

Instalación de VaN3Twin en Docker

Guía detallada paso a paso para macOS Tahoe sobre Apple Silicon

Repositorio: <https://github.com/DriveX-devs/VaN3Twin>

Pablo Barbecho

Universidad de Cuenca

Agosto 2026

Resumen

VaN3Twin (evolución del framework *ms-van3t* del Politecnico di Torino / DriveX) es un conjunto de módulos para ns-3 que permite construir y simular aplicaciones VANET/V2X conformes con ETSI, acoplando el simulador de red ns-3 con el simulador de movilidad SUMO mediante TraCI. El proyecto **no** ofrece soporte oficial de Docker: su instalación nativa asume Ubuntu (20.04/22.04/24.04) y un script interactivo (`sandbox_builder.sh`). Este documento analiza el software y su proceso de instalación, y presenta una contenedorización completa y reproducible para Docker Desktop en macOS Tahoe con Apple Silicon (arm64): un `Dockerfile` explicado capa por capa, un `docker-compose.yml` comentado directiva por directiva, el soporte de interfaz gráfica (`sumo-gui`) vía VNC integrado (Xvfb + x11vnc, con XQuartz como alternativa), y las implicaciones técnicas de cada decisión.

Índice de contenidos

1	Análisis del software VaN3Twin	2
1.1	Qué es y de dónde viene	2
1.2	Estructura del repositorio	2
1.3	Cómo se instala nativamente (y por qué importa para Docker)	2
2	Estrategia de contenedorización para macOS Apple Silicon	5
2.1	Cómo funciona Docker en un Mac	5
2.2	Decisiones de diseño y sus implicaciones	5
3	Prerrequisitos en el Mac (macOS Tahoe)	7
3.1	Software necesario	7
3.2	Configurar recursos de Docker Desktop	7
3.3	(Opcional) Configurar XQuartz — solo para la alternativa X11	7
3.4	Flujo general de la instalación	8
4	El Dockerfile, capa por capa	9
4.1	Cabecera y argumentos	9
4.2	Capa 1 — Toolchain y dependencias	9
4.3	Capa 2 — SUMO	10
4.4	Capa 3 — gRPC desde código fuente	10
4.5	Capa 4 — Usuario no root	10
4.6	Capas 5 y 6 — Clonado y sandbox	11
4.7	Capa 7 — Configuración y compilación de ns-3	11
4.8	Capas 3b y 3c — Visor NetAnim y soporte PyViz	11
4.9	Cierre	11

5	El docker-compose.yml, directiva por directiva	12
5.1	Servicio e imagen	12
5.2	Interactividad y entorno	12
5.3	Volúmenes: la decisión más importante	12
5.4	Puertos, recursos y reinicio	13
6	Construcción y primer arranque, paso a paso	14
6.1	Preparar la carpeta de trabajo	14
6.2	Construir la imagen	14
6.3	Arrancar y entrar	14
6.4	Verificación rápida (smoke test)	14
7	Ejecutar simulaciones	15
7.1	Sin interfaz gráfica (headless)	15
7.2	Con GUI: VNC (método por defecto)	15
7.3	Alternativa: GUI vía XQuartz (X11 en red)	15
7.4	Visualizador web de vehículos	16
7.5	Flujo de trabajo de desarrollo	16
7.6	Ciclo de vida del contenedor	16
8	Visualización de mensajes y de la simulación	17
8.1	Wireshark sobre los .pcap: el contenido de cada mensaje	17
8.2	NetAnim: animación de paquetes entre nodos	17
8.3	PyViz: vista interactiva en vivo	19
8.4	PyViz vs. NetAnim: ¿cuál usar?	19
8.5	vehicle-visualizer: mapa web de vehículos	19
9	Sionna sin GPU: servidor nativo en el Mac	21
9.1	Análisis: ¿se puede usar Sionna sin GPU NVIDIA?	21
9.2	Arquitectura de la solución nativa	21
9.3	Instalación (una sola vez)	21
9.4	Ejecución	22
9.5	Expectativas de rendimiento en CPU	23
10	Solución de problemas	25
11	Referencias	26
A	setup-sionna-macos.sh (Sionna nativo, instalación)	27
B	run-sionna-server.sh y run-sionna-sim.sh (Sionna nativo, ejecución)	29

1 Análisis del software VaN3Twin

1.1 Qué es y de dónde viene

VaN3Twin es el nuevo nombre del framework *ms-van3t*; integra todas sus funcionalidades y añade otras nuevas (p.ej. la integración con el ray-tracer NVIDIA Sionna). Es un conjunto de módulos para **ns-3** (versión **ns-3-dev** del fork 5G-LENA de CTTC, con soporte del módulo NR-V2X) que permite simular comunicaciones vehiculares V2X con pila ETSI ITS-G5 completa (mensajes CAM, DENM, CPM, VAM, IVIM, en versiones v1 y v2 con ASN.1 real), pudiendo conmutar fácilmente la tecnología de acceso: IEEE 802.11p, LTE, C-V2X Modo 4 y NR-V2X. La movilidad de los vehículos proviene de **SUMO** (versiones probadas: 1.6.0 a 1.18.0), acoplado en tiempo de simulación mediante **TraCI**. Licencia GPL-2.0.

1.2 Estructura del repositorio

Los componentes principales dentro de **src/** son:

Módulo	Función
automotive	Pila ETSI ITS-G5 (CAM/DENM/CPM/VAM, ASN.1, DCC, facilities), aplicaciones de ejemplo y mapas SUMO. Requiere OpenSSL y OpenCV (<code>find_package(OpenCV REQUIRED)</code>).
traci , traci-applications	Lógica de acoplamiento ns-3 ↔ SUMO (cliente TraCI).
cv2x	Modelo C-V2X (LTE-V2X) en modo de transmisión 4.
nr	Módulo 5G-LENA con NR-V2X (se descarga aparte, rama nr-v2x-dev).
gps-tc	Reproducción de trazas GPS reales como movilidad.
carla	Cliente gRPC hacia CARLA/OpenCDA (percepción y movilidad). Requiere gRPC con archivos CMake (<code>find_package(gRPC CONFIG REQUIRED)</code>).
sionna	Integración con el ray-tracer NVIDIA Sionna (canal radio realista).
vehicle-visualizer	Visualizador web (Node.js) de los vehículos: servidor HTTP + socket UDP interno 48110.

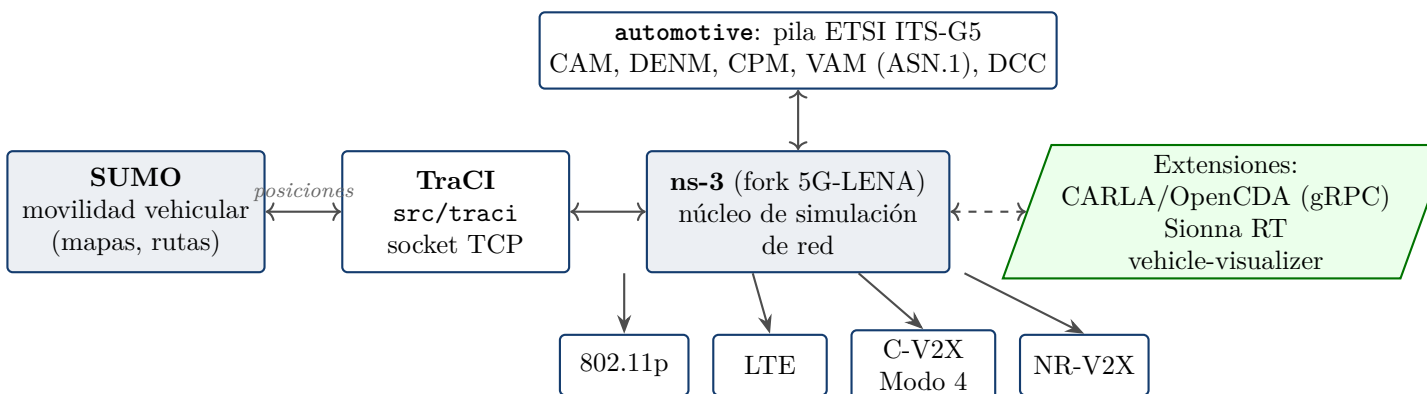


Figura 1: Arquitectura funcional de VaN3Twin: SUMO aporta la movilidad, TraCI la sincroniza con ns-3, y la pila ETSI ITS-G5 se ejecuta sobre la tecnología de acceso elegida.

1.3 Cómo se instala nativamente (y por qué importa para Docker)

El README oficial define este procedimiento sobre Ubuntu: instalar SUMO; clonar el repo; ejecutar `./sandbox_builder.sh` (con `install-dependencies` la primera vez); y finalmente `./ns3 configure + ./ns3 build` dentro de `ns-3-dev/`. El script `sandbox_builder.sh` tiene varias particularidades que **condicionan el diseño del Dockerfile**:

1. **Rechaza ejecutarse como root** (`if [ "$EUID" -eq 0 ] ... exit 1`). En Docker el usuario por defecto es root, así que hay que crear un usuario sin privilegios.
2. **Es interactivo**: contiene `read -p "Press ENTER to continue..."`. En una build no interactiva hay que alimentarle la entrada (`echo "" | ./sandbox_builder.sh`).
3. **Detecta la versión de Ubuntu** con `lsb_release` y solo acepta 20.04, 22.04 o 24.04. La imagen base debe ser una de ellas y tener instalado `lsb-release`.
4. Con `install-dependencies` compila **gRPC v1.60.0** y **OpenCV desde código fuente** (horas de CPU). En Docker conviene separar esto en capas cacheables y sustituir OpenCV por el paquete `apt`.
5. Clona el ns-3 de CTTC (etiqueta `ns-3-dev-v2x-v0.2`) y el módulo `nr` (rama `nr-v2x-dev`), copia los módulos de VaN3Twin dentro de `ns-3-dev/src/`, aplica parches (modelos de propagación extendidos, SignalInfo, ficheros Sionna) y **se auto-elimina** (`rm $0`) al terminar.
6. En Ubuntu 22.04+ el `configure` exige `-disable-werror`; en 24.04 además `CXXFLAGS="-include cstdint"`.

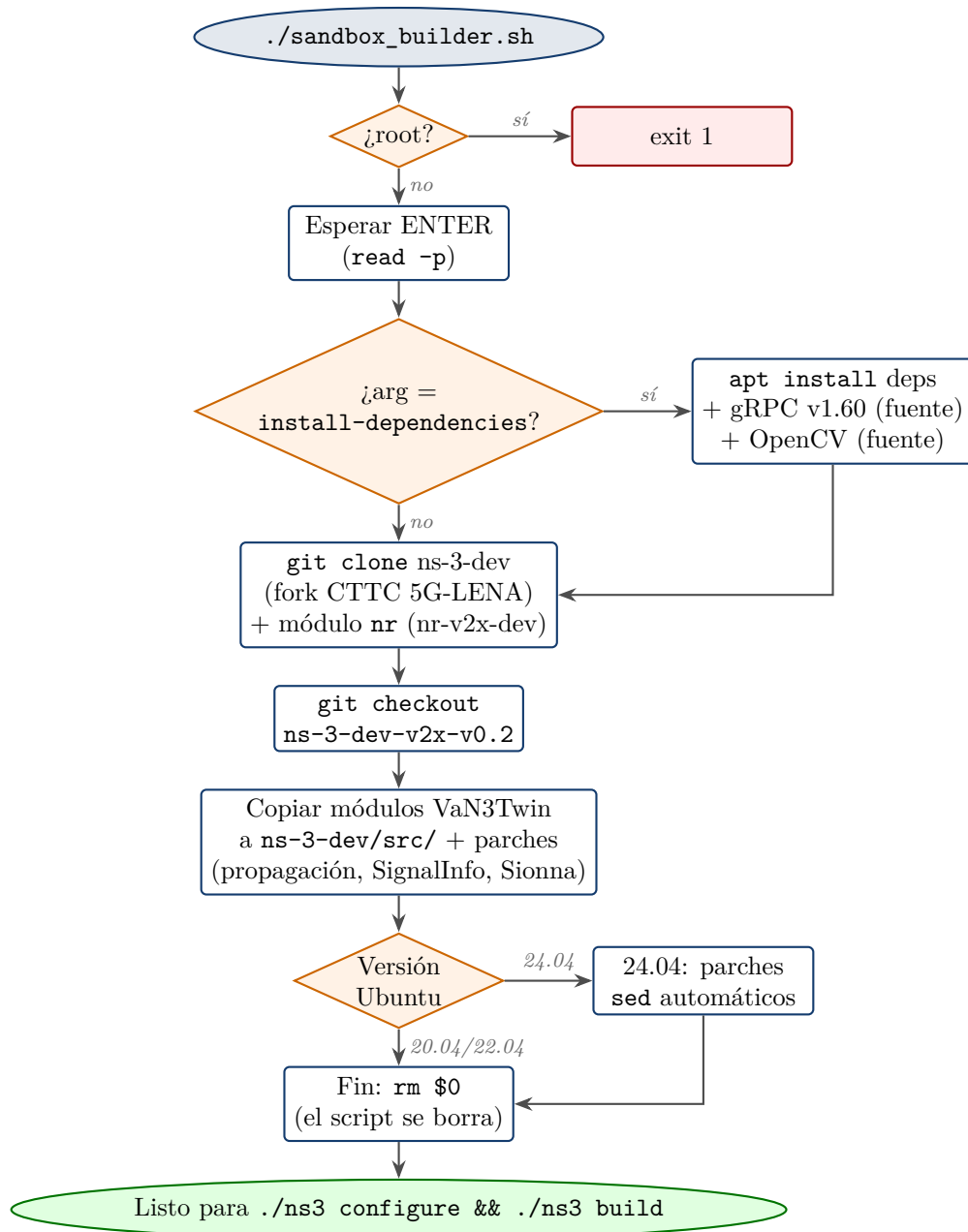


Figura 2: Flujo interno de `sandbox_builder.sh`. Las decisiones marcadas condicionan el diseño del Dockerfile (usuario no root, entrada no interactiva, `lsb-release`, dependencias pre-instaladas).

2 Estrategia de contenedorización para macOS Apple Silicon

2.1 Cómo funciona Docker en un Mac

En macOS, Docker Desktop no ejecuta contenedores “directamente”: crea una **máquina virtual Linux ligera** (Virtualization.framework) y todos los contenedores viven dentro de esa VM. Esto tiene tres implicaciones directas para VaN3Twin:

Implicación

- **Recursos:** la RAM/CPU/disco que ve el contenedor son los asignados a la VM en *Docker Desktop* → *Settings* → *Resources*, no los del Mac. Compilar ns-3 con 8 hilos puede requerir 10–16 GB de RAM de la VM.
- **E/S de ficheros:** los *bind mounts* (carpetas del Mac montadas en el contenedor) pasan por VirtioFS y son más lentos que el disco interno de la VM. Aun así, este proyecto monta el árbol completo de VaN3Twin como bind mount (`./VaN3Twin`) para tenerlo visible en el Finder y editable con VS Code desde el Mac; el coste se nota sobre todo en builds completos de ns-3 (ediciones y simulaciones apenas lo acusan).
- **Red:** no hay acceso directo a las interfaces físicas del Mac (ni `macvlan`); el modo emulación de VaN3Twin (`v2x-emulator` sobre una NIC real) no es viable en este entorno.

2.2 Decisiones de diseño y sus implicaciones

Decisión	Justificación e implicación
Base ubuntu:22.04	Es una de las tres versiones aceptadas por <code>sandbox_builder.sh</code> ; existe imagen arm64 oficial; evita los parches extra de 24.04. Obliga a usar <code>-disable-werror</code> .
Arquitectura linux/arm64	Nativa en Apple Silicon: sin emulación Rosetta, compilación 2–5× más rápida. Todo se construye desde fuente o desde paquetes multi-arch, así que no hay dependencias x86_64.
SUMO 1.12.0 desde repos Ubuntu (no PPA)	Los PPA de Launchpad (<code>ppa:sumo/stable</code>) normalmente no publican binarios arm64 ⇒ fallaría en Apple Silicon. Además 1.12.0 está dentro del rango probado (1.6–1.18) y el README advierte contra versiones más nuevas de TraCI.
gRPC v1.60.0 desde fuente	Obligatorio: <code>src/carla</code> usa <code>find_package(gRPC CONFIG REQUIRED)</code> y el paquete apt de 22.04 no trae los ficheros CMake de gRPC. Añade ~30–45 min de build (una sola vez, queda cacheado en una capa).
OpenCV desde apt (<code>libopencv-dev</code>)	<code>src/automotive</code> solo necesita <code>find_package(OpenCV)</code> ; OpenCV 4.5.4 de Ubuntu lo satisface. Evita ~40 min de compilación y ~2 GB de imagen frente al procedimiento del script oficial.
Sin <code>libc6-dev-i386</code>	Ese paquete solo existe en amd64; en arm64 rompería el <code>apt install</code> . No es necesario para compilar VaN3Twin.
Usuario <code>vanet</code> no root	<code>sandbox_builder.sh</code> aborta si detecta root. Además es buena práctica de seguridad; se le da <code>sudo</code> sin contraseña para comodidad en desarrollo.
Build completo dentro de la imagen	La imagen queda “lista para simular” (<code>docker compose up</code> y ejecutar). Contrapartida: imagen grande (~10–15 GB) y build inicial largo (1–3 h).
Volumen nombrado sobre el árbol ns-3	Persiste ediciones y recompilaciones entre recreaciones del contenedor con E/S rápida (disco de la VM).
GUI vía VNC (Xvfb + x11vnc en el contenedor)	<code>sumo-gui</code> necesita un servidor X con GLX; el X virtual interno lo aporta por software (llvmpipe) sin depender del Mac. Visor: <i>Compartir pantalla</i> de macOS (<code>vnc://127.0.0.1:5901</code>). XQuartz queda como alternativa opcional.

Advertencia

Limitaciones en Mac/Docker que conviene conocer desde el principio:

- **Extensión CARLA/OpenCDA:** CARLA 0.9.12 se distribuye solo como binario x86_64 y recomienda GPU NVIDIA ≥ 8 GB. No es ejecutable de forma útil en Apple Silicon (ni con Rosetta). El módulo `carla` de `ns-3` *se compila* igualmente (por eso se instala gRPC), pero el simulador CARLA debería correr en una máquina Linux x86_64 remota.
- **Sionna:** la GPU NVIDIA *no* es obligatoria — el ray tracer funciona en CPU (backend LLVM de Dr.Jit). La sección 9 detalla la solución recomendada sin GPU: servidor Sionna **nativo en el Mac** conectado al contenedor por UDP. Para estudios grandes sigue siendo preferible una máquina remota con GPU.
- **Modo emulación** (`v2x-emulator`): requiere una NIC real; la VM de Docker Desktop no la expone.

Nada de esto afecta al uso principal: **simulaciones V2X completas (802.11p, LTE, C-V2X, NR-V2X) con SUMO, incluida la GUI.**

3 Prerrequisitos en el Mac (macOS Tahoe)

3.1 Software necesario

Listing 1: Instalación de prerrequisitos con Homebrew

```
# 1. Docker Desktop (runtime de contenedores con VM Linux integrada)
brew install --cask docker

# 2. Para la GUI de SUMO NO se necesita nada más: va por VNC y el visor
#    ("Compartir pantalla") viene integrado en macOS.
#    XQuartz solo si prefieres la alternativa X11 (ver 3.3):
# brew install --cask xquartz

# 3. Verificaciones
docker --version          # Docker version 27.x o superior
docker compose version    # Docker Compose version v2.x
uname -m                  # arm64 (Apple Silicon)
```

3.2 Configurar recursos de Docker Desktop

En *Docker Desktop* → *Settings* → *Resources*:

Recurso	Mínimo	Recomendado
CPUs	4	6–8
Memoria	8 GB	12–16 GB
Swap	1 GB	2 GB
Disco virtual	40 GB	60–80 GB

Implicación

La compilación de ns-3 lanza un proceso `g++` por núcleo y algunos ficheros ASN.1 del módulo `automotive` consumen 1–2 GB de RAM cada uno al compilarse con `-O2`. Si la VM tiene poca memoria, el kernel de la VM matará procesos (`g++: internal compiler error: Killed`) y la build fallará. Regla práctica: RAM de la VM $\geq 1.5 \times$ CPUs asignadas en GB.

3.3 (Opcional) Configurar XQuartz — solo para la alternativa X11

Con la GUI por VNC (método por defecto, §7.2) esta sección no es necesaria: no hay nada que instalar ni configurar en el Mac. Sigue estos pasos únicamente si decides usar la variante XQuartz (X11 en red), que requiere además activar las líneas comentadas `DISPLAY/LIBGL_ALWAYS_INDIRECT` del compose.

1. Abrir XQuartz, menú *XQuartz* → *Settings* → *Security* y marcar “**Allow connections from network clients**”.
2. Habilitar el **GLX indirecto** (XQuartz lo trae desactivado por defecto; sin él, `sumo-gui` abre la ventana pero el canvas OpenGL falla con `X Error: GLXBadContext` y el mapa no se dibuja):

```
defaults write org.xquartz.X11 enable_iglx -bool true
```

3. Cerrar XQuartz por completo y volver a abrirlo (o reiniciar el Mac) para aplicar ambos cambios.
4. Antes de cada sesión con GUI, autorizar al cliente local:


```
/opt/X11/bin/xhost + 127.0.0.1
```

la VM de Docker Desktop llega al host vía NAT y aparece como 127.0.0.1

Advertencia

`xhost + 127.0.0.1` permite que *cualquier* proceso local se conecte a tu pantalla X11 mientras XQuartz esté abierto. Es aceptable en un equipo personal; revócalo al terminar con `xhost - 127.0.0.1` o cerrando XQuartz.

3.4 Flujo general de la instalación

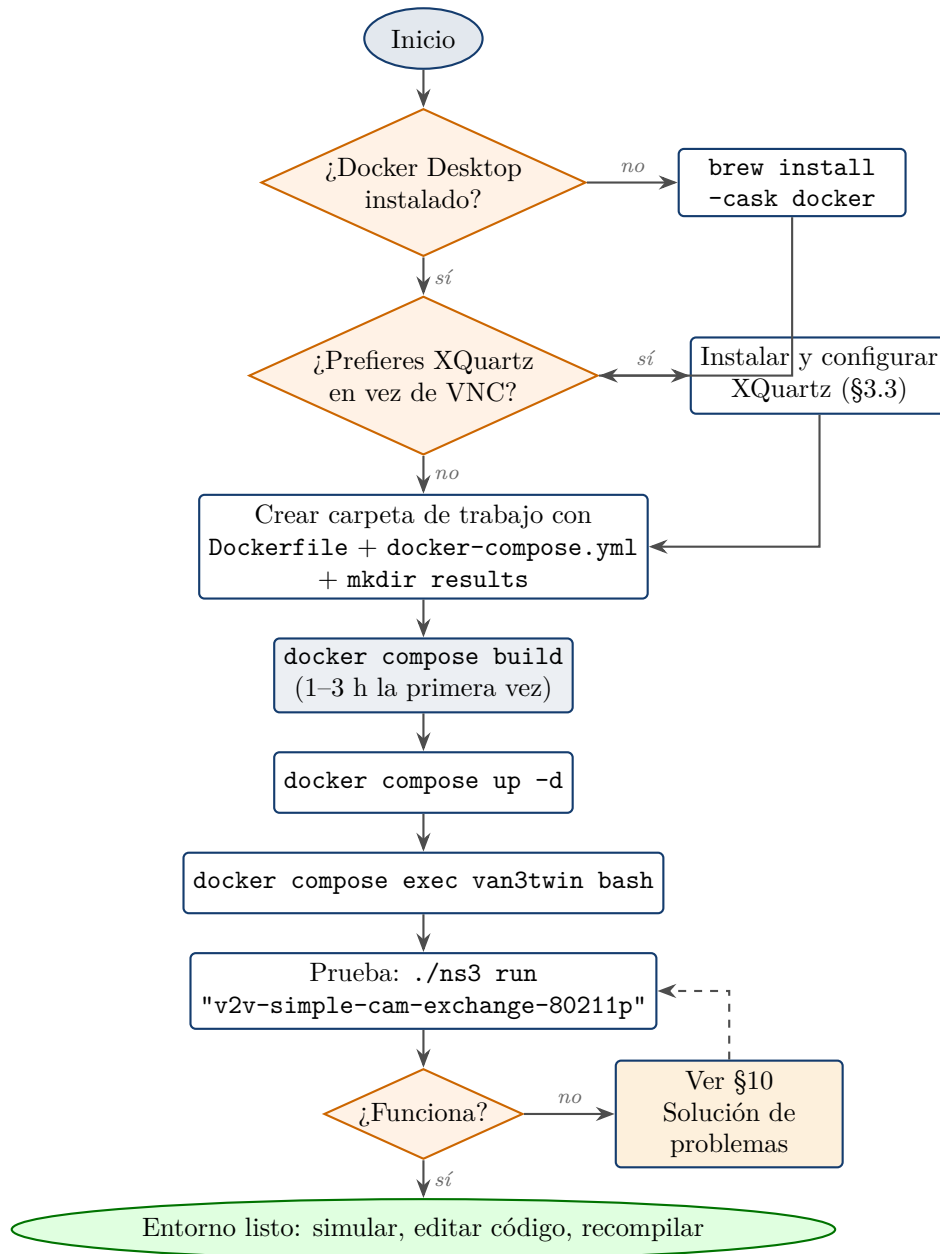


Figura 3: Flujo general de la instalación de VaN3Twin en Docker sobre macOS.

4 El Dockerfile, capa por capa

El código de la imagen vive únicamente en el fichero **Dockerfile** de la carpeta de trabajo (fuente de verdad; no se duplica aquí para evitar desincronización). Esta sección explica cada capa y sus implicaciones. La figura 4 resume el pipeline.

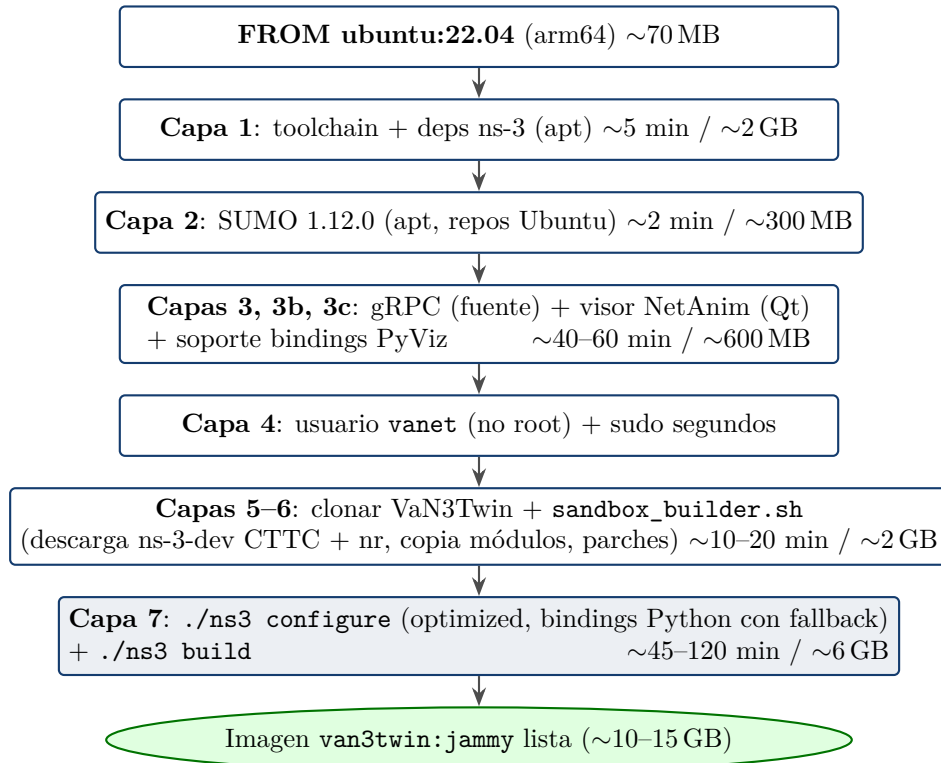


Figura 4: Pipeline de capas del Dockerfile con tiempos orientativos en un M2/M3 con 6 CPUs y 12 GB asignados a Docker. Cada capa se cachea: si algo falla o se cambia, solo se reconstruye desde la capa modificada hacia abajo.

4.1 Cabecera y argumentos

Implicación

DEBIAN_FRONTEND=noninteractive evita que paquetes como **tzdata** detengan la build pidiendo la zona horaria por teclado (clásico bloqueo de builds Docker con Ubuntu). Al declararlo como ARG (y no ENV) solo aplica durante la construcción, no queda “pegado” en los contenedores. **GRPC_VERSION** y **VAN3TWIN_REF** permiten fijar versiones reproducibles desde el **docker-compose.yml** sin editar el Dockerfile.

4.2 Capa 1 — Toolchain y dependencias

La lista replica la de **sandbox_builder.sh install-dependencies** con cuatro ajustes razonados:

- **Se omite libc6-dev-i386**: no existe para arm64; su inclusión rompería la build en Apple Silicon. Solo sirve para compilar binarios de 32 bits en x86, algo que VaN3Twin no necesita.
- **Se añade libopencv-dev** en lugar de compilar OpenCV desde fuente (ver §2.2).
- **Se añaden libgl1-mesa-dri y mesa-utils**: el driver software de Mesa (**swrast/llvmpipe**) permite que **sumo-gui** renderice OpenGL *dentro* del contenedor con **LIBGL_ALWAYS_SOFTWARE=1**,

sin depender del GLX de XQuartz. Sin este paquete se produce `libGL error: failed to load driver: swrast` y una lluvia de `GLXBadContext`. `mesa-utils` aporta `glxinfo/glxgears` para diagnosticar.

- **Se omiten los paquetes de bindings Python de ns-3** (`gir1.2-goocanvas`, `python3-pygraphviz`, etc.) porque el `configure` usa `-disable-python`.

Implicación

`-no-install-recommends` reduce cientos de MB de paquetes sugeridos. `rm -rf /var/lib/apt/lists/*` en el *mismo* RUN borra los índices de apt antes de que la capa se selle: si se hiciera en otro RUN, el espacio ya estaría consumido en la capa anterior (las capas son inmutables y aditivas). `lsb-release` y `sudo` no son opcionales: el primero lo exige la detección de versión de `sandbox_builder.sh`; el segundo da flexibilidad al usuario no root.

4.3 Capa 2 — SUMO

Implicación

Ubuntu 22.04 empaqueta SUMO **1.12.0** para arm64 y amd64, dentro del rango probado por los autores (1.6.0–1.18.0). El README oficial sugiere el PPA `ppa:sumo/stable`, pero (a) los PPA rara vez publican arm64, y (b) instalaría un SUMO más nuevo cuya API TraCI podría no ser compatible (“*Back compatibility is not ensured with new versions of TraCI*”). `SUMO_HOME` es leído por las herramientas de SUMO (p.ej. `randomTrips.py`, `netconvert`) y por scripts del framework. El paquete `sumo` incluye `sumo-gui`.

4.4 Capa 3 — gRPC desde código fuente

Implicación

Es la capa “cara” que no se puede evitar: `src/carla/CMakeLists.txt` ejecuta `find_package(gRPC CONFIG REQUIRED)`, que busca los ficheros `gRPCConfig.cmake` — el paquete `libgrpc++-dev` de Ubuntu 22.04 **no los incluye**, de ahí que el script oficial también compile gRPC v1.60.0 desde fuente. Detalles del comando: `-depth 1` y `-init -depth 1` clonan solo el último commit (ahorra ~1GB de descarga); `-DgRPC_BUILD_TESTS=OFF` recorta mucho tiempo; y el `rm -rf /opt/grpc` final, al estar en el mismo RUN, evita que ~6 GB de fuentes y objetos queden fosilizados en la capa. `ldconfig` refresca la caché del enlazador para que `/usr/local/lib` sea visible.

4.5 Capa 4 — Usuario no root

Implicación

Sin esta capa, `sandbox_builder.sh` abortaría con “*Please do NOT run this script as root*”. A partir del USER `vanet`, todas las instrucciones siguientes (y la shell del contenedor en ejecución) corren como usuario normal: los ficheros creados en los volúmenes tendrán un propietario coherente y se reduce la superficie de ataque. El `sudo` sin contraseña es un compromiso cómodo para un entorno de *desarrollo*; en producción se eliminaría.

4.6 Capas 5 y 6 — Clonado y sandbox

Implicación

El `echo ""` canalizado responde al `read -p "Press ENTER..."` del script, convirtiéndolo en no interactivo. Se ejecuta **sin** el argumento `install-dependencies` porque las dependencias ya están en las capas 1–3 (la rama `install-dependencies` del script usa `sudo apt` y compilaría OpenCV/gRPC otra vez). Tras esta capa, el árbol queda en `/home/vanet/VaN3Twin/ns-3-dev` con los módulos de VaN3Twin integrados en `src/`; el script borra los ficheros originales del repo raíz y se auto-elimina. Nota: esta capa descarga ~1–2 GB desde GitLab (`ns-3-dev` de CTTC) — requiere red durante el build.

4.7 Capa 7 — Configuración y compilación de ns-3

Implicación

Cada flag tiene una razón documentada:

- `-build-profile=optimized`: recomendado por el README para acelerar el tiempo de ejecución de las simulaciones. Para depurar con `gdb` conviene reconfigurar con `debug` (se puede hacer después, dentro del contenedor).
- `-enable-examples`: sin él **no se compilan** los ejemplos `v2v-*/v2i-*`, que son la forma principal de usar VaN3Twin.
- `-disable-python` (por defecto, vía ARG `NS3_PYTHON_FLAG`): los bindings Python solo servirían para PyViz y está verificado que no compilan con los parches de VaN3Twin (§8.3); el ARG permite reintentarlo, con *fallback* automático a sin-Python.
- `-disable-werror`: **obligatorio en Ubuntu 22.04+** según el README; sin él, warnings del compilador moderno se tratan como errores y la build falla.

`./ns3 build` usa todos los núcleos disponibles; el paralelismo real lo limita la directiva `cpus` del `compose` / los recursos de la VM. Hacer el build *dentro* de la imagen significa que cualquier `docker run` de esta imagen ya contiene un ns-3 compilado: la imagen es autosuficiente y distribuable (p.ej. a estudiantes) vía `docker save/docker push`.

4.8 Capas 3b y 3c — Visor NetAnim y soporte PyViz

Dos capas pequeñas dejan las herramientas de visualización listas sin scripts adicionales. La **capa 3b** clona y compila el visor **NetAnim** (aplicación Qt5, inexistente en los repos de Ubuntu) y lo instala en `/usr/local/bin/NetAnim`. La **capa 3c** instala el soporte de bindings Python (`pybindgen`; `cppy` solo si hay wheel binario) por si upstream arregla los bindings algún día — hoy PyViz no compila con los parches de VaN3Twin (§8.3). Además, la **capa 6b** parchea el ejemplo `v2v-simple-cam-exchange-80211p` con los tres cambios de NetAnim (§8.2), de modo que la animación funciona de fábrica.

4.9 Cierre

Implicación

`DISPLAY=:99` y `LIBGL_ALWAYS_SOFTWARE=1` apuntan por defecto al X virtual interno con OpenGL por software (la GUI VNC, §7.2); el `compose` puede sobrescribirlos para la alternativa XQuartz. `CMD ["/bin/bash"]` define el proceso por defecto: una shell interactiva (el `compose` la mantiene viva con `tty/stdin_open`).

5 El docker-compose.yml, directiva por directiva

El código vive únicamente en el fichero `docker-compose.yml` de la carpeta de trabajo; aquí se explica cada directiva. Con `docker compose` la ejecución se reduce a `build / up / exec / down`, sin memorizar comandos `docker run` largos.

5.1 Servicio e imagen

Implicación

`build.context:` `.` indica que el Dockerfile y su contexto están en la misma carpeta que el `compose`. `args` inyecta los ARG del Dockerfile: cambiar `VAN3TWIN_REF` a una etiqueta concreta congela la versión del framework (reproducibilidad para un paper). `image` da nombre a la imagen resultante; `platform:` `linux/arm64` hace explícito el modo nativo en Apple Silicon — si algún día se necesitara `linux/amd64`, Docker Desktop lo emularía con Rosetta (funciona, pero la compilación sería varias veces más lenta). `container_name` fija un nombre estable para `docker compose exec van3twin ....`

5.2 Interactividad y entorno

Implicación

`tty + stdin_open` equivalen a `docker run -it`: sin ellos, el `bash` terminaría al instante (no habría terminal adjunta) y el contenedor moriría nada más arrancar. El `command` arranca automáticamente, *en cada inicio del contenedor*, el servidor X virtual (`Xvfb :99`, con `+extension GLX` para anunciar OpenGL correctamente), el gestor de ventanas ligero `fluxbox` (sin él, las ventanas aparecen sin bordes e inmóviles sobre fondo negro) y el VNC (`x11vnc`, puerto 5900) antes de dejar la shell como proceso principal: la GUI por VNC (§7.2) queda disponible sin pasos manuales (`Xvfb/x11vnc` son procesos, no configuración: mueren al parar el contenedor, por eso se lanzan aquí). `working_dir` hace que cualquier `exec` aterrice directamente en `ns-3-dev`, donde se ejecuta `./ns3`. El entorno por defecto apunta al X virtual con render por software (`DISPLAY=:99 + LIBGL_ALWAYS_SOFTWARE=1`, la combinación más robusta en macOS); las líneas comentadas restauran la variante XQuartz sin reconstruir la imagen (solo `docker compose up -d`).

5.3 Volúmenes: la decisión más importante

Implicación

Se usan dos *bind mounts* (carpetas reales del Mac, visibles en Finder):

- `./VaN3Twin` sobre `/home/vanet/VaN3Twin`: todo el código y el build de ns-3 en una carpeta normal del Mac, editable con VS Code (code `~/van3twin-docker/VaN3Twin`). OJO: a diferencia de un volumen nombrado, Docker **no la puebla automáticamente**; antes del *primer* arranque hay que copiarle el contenido de la imagen: `mkdir -p VaN3Twin && docker run -rm -v "$PWD/VaN3Twin":/dst van3twin:jammy sh -c "cp -a /home/vanet/VaN3Twin/. /dst/"`. A partir de ahí, todo cambio — código editado, recompilaciones, resultados intermedios — **persiste siempre**, incluso si se eliminan el contenedor y la imagen.
- `./results`: para copiar los `.csv`, `.pcap` o trazas que genere una simulación y analizarlos con herramientas del Mac.

Advertencia

Efecto colateral del bind mount: si **reconstruyes la imagen** (`docker compose build`), la carpeta `./VaN3Twin` sigue *tapando* el contenido nuevo de la imagen en esa ruta. Para estrenar lo recién horneado: renombra la carpeta (queda como respaldo), vuélvela a poblar desde la imagen con el comando anterior y recupera tus cambios propios desde el respaldo (o con `git`).

5.4 Puertos, recursos y reinicio

Implicación

- **ports:** publica el servidor web del `vehicle-visualizer` (Node.js) para abrirlo en el navegador del Mac (`http://localhost:8080`). El socket UDP 48110 que usa el visualizador internamente **no** se publica: ns-3 y el servidor Node conviven dentro del mismo contenedor.
- **shm_size:** `/dev/shm` por defecto es de 64 MB en Docker; Qt (`sumo-gui`) y OpenCV usan memoria compartida y pueden agotarlo, provocando cierres críticos de la GUI.
- **mem_limit** y **cpus:** topes del contenedor, *dentro* de lo asignado a la VM. Sirven para que una compilación o una simulación grande no congele el resto del sistema. Ajusta según tu Mac (§3.2).
- **restart:** `"no"`: es un contenedor interactivo de desarrollo; no tiene sentido resucitarlo automáticamente tras un reinicio del Mac.

6 Construcción y primer arranque, paso a paso

6.1 Preparar la carpeta de trabajo

```
mkdir -p ~/van3twin-docker/results
cd ~/van3twin-docker
# Copiar aquí los ficheros de esta guía:
#   Dockerfile
#   docker-compose.yml
#   setup-sionna-macos.sh (opcional, Sionna nativo sin GPU)
#   run-sionna-server.sh (opcional, Sionna nativo sin GPU)
#   run-sionna-sim.sh (opcional, Sionna nativo sin GPU)
chmod +x setup-sionna-macos.sh run-sionna-*.sh
```

6.2 Construir la imagen

```
# Con registro detallado (recomendado la primera vez):
docker compose build --progress=plain 2>&1 | tee build.log

# Seguimiento en otra terminal:
docker stats
```

Nota

Duración típica en un M2/M3 con 6 CPUs y 12 GB para la VM: **1,5–3 horas** (gRPC ~40 min, ns-3 ~60–90 min). La descarga total ronda los 3–4 GB. Si la build se interrumpe (red, sleep del Mac), al relanzarla continúa desde la última capa cacheada — esa es la ventaja de haber separado el proceso en capas.

6.3 Arrancar y entrar

```
docker compose up -d          # crea el contenedor (ANTES del primer arranque: poblar
    ↪ ./VaN3Twin, ver seccion de volúmenes)
docker compose ps             # estado: van3twin running
docker compose exec van3twin bash # shell dentro del contenedor
```

6.4 Verificación rápida (smoke test)

Dentro del contenedor:

```
# 1. Entorno
whoami          # vanet
lsb_release -ds # Ubuntu 22.04.x LTS
uname -m        # aarch64
sumo --version  # Eclipse SUMO Version 1.12.0
echo $SUMO_HOME # /usr/share/sumo

# 2. ns-3 responde
./ns3 show profile # build profile: optimized

# 3. Simulación V2V sin GUI (80211p, intercambio de CAMs)
./ns3 run "v2v-simple-cam-exchange-80211p"
```

Si el paso 3 imprime estadísticas de CAMs transmitidos/recibidos al terminar, la instalación es funcional.

7 Ejecutar simulaciones

7.1 Sin interfaz gráfica (headless)

Muchos ejemplos activan la GUI de SUMO por defecto (`bool sumo_gui = true`); para ejecutarlos sin GUI hay que desactivarla explícitamente:

```
./ns3 run "v2v-emergencyVehicleAlert-80211p --sumo-gui=false"
./ns3 run "v2i-areaSpeedAdvisor-80211p --sumo-gui=false"
./ns3 run "v2v-emergencyVehicleAlert-nrv2x --sumo-gui=false"
./ns3 run "v2v-coexistence-80211p-nrv2x"
```

7.2 Con GUI: VNC (método por defecto)

La GUI de SUMO funciona por VNC sin instalar nada en el Mac: el `command` del compose arranca en cada inicio del contenedor un servidor X virtual (`Xvfb`, `display :99`) — que expone visuales GLX por software (Mesa `llvmpipe`) — y `x11vnc` lo publica por VNC. El visor es la app integrada de macOS *Compartir pantalla*. El puerto 5900 del contenedor se publica como **5901** en el Mac (solo 127.0.0.1), porque el 5900 local suele estar ocupado por el propio servicio *Compartir pantalla* de macOS.

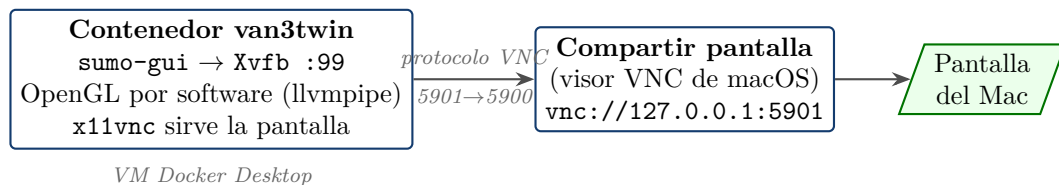


Figura 5: Camino de la GUI por VNC: todo el pipeline gráfico (X + OpenGL) corre dentro del contenedor y el Mac solo muestra el resultado.

Uso diario (dos pasos):

Listing 2: En el Mac

```
open vnc://127.0.0.1:5901          # o Finder -> Cmd+K; dejar la ventana abierta
docker compose exec van3twin bash
```

Listing 3: Dentro del contenedor

```
sumo-gui &          # prueba: la ventana de SUMO aparece en el visor VNC
./ns3 run "v2v-emergencyVehicleAlert-80211p"  # GUI activa por defecto
```

Implicación

Al correr todo el pipeline gráfico dentro del contenedor, esta ruta no depende de servidores X externos ni de GLX del Mac: es la opción más robusta (y suele ser más fluida que X11 sobre TCP), a costa de algo más de CPU. `Xvfb` y `x11vnc` son procesos: mueren al parar el contenedor y por eso los relanza el `command` en cada arranque, sin pasos manuales. Todo lo necesario (`Xvfb`, `fluxbox`, `x11vnc`, `Mesa/llvmpipe`, `NetAnim`) viene horneado en la imagen: no se requiere ningún script auxiliar.

7.3 Alternativa: GUI vía XQuartz (X11 en red)

Si prefieres ventanas nativas de X11 en el escritorio del Mac (sin visor VNC), existe la variante XQuartz. Requiere: instalar y configurar XQuartz (§3.3), y activar en el compose las líneas comentadas (`DISPLAY: host.docker.internal:0` y `LIBGL_ALWAYS_INDIRECT: "1"`, comentando a la vez `DISPLAY: ":99"` y `LIBGL_ALWAYS_SOFTWARE`) seguido de `docker compose up -d`.

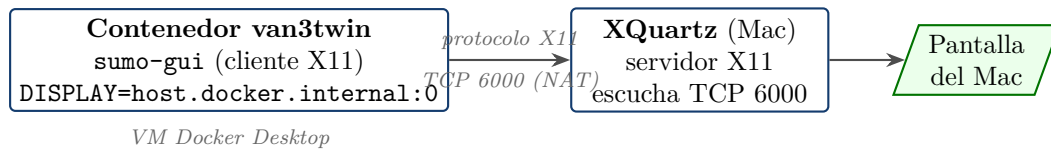


Figura 6: Camino de la GUI en la alternativa XQuartz: sumo-gui dibuja en la pantalla del Mac a través del socket TCP de XQuartz.

Listing 4: En el Mac

```
open -a XQuartz
/opt/X11/bin/xhost + 127.0.0.1
docker compose exec van3twin bash
```

Advertencia

Esta variante depende del GLX de XQuartz (`enable_iglx`) y es la fuente de los errores `GLXBadContext` / `No matching fbConfigs` documentados en §10. Si aparecen y no se resuelven, volver al método VNC por defecto (§7.2), que no depende de XQuartz.

7.4 Visualizador web de vehículos

Algunos ejemplos integran el `vehicle-visualizer` (flag `-vehicle-visualizer=true`); con el puerto 8080 ya publicado en el compose, basta abrir `http://localhost:8080` en el navegador del Mac. Requisitos y detalles en §8.5; para visualizar el *intercambio de mensajes* (Wireshark, NetAnim, PyViz), ver la sección 8.

7.5 Flujo de trabajo de desarrollo

```
# Editar código (dentro del contenedor, o con VS Code + extensión Dev Containers
# adjuntada al contenedor 'van3twin'):
nano src/automotive/examples/v2v-simple-cam-exchange-80211p.cc

# Recompilar (incremental, solo lo cambiado):
./ns3 build

# Exportar resultados al Mac:
cp mi-traza.csv ~/results/          # aparece en ~/van3twin-docker/results
```

Nota

Los cambios de código y los builds incrementales quedan en la carpeta `./VaN3Twin` del Mac: sobreviven a `docker compose down`, a reinicios e incluso a borrar el contenedor y la imagen. Para versionar tu trabajo, git normal desde el Mac (la carpeta es local); también puedes abrirla directamente en VS Code.

7.6 Ciclo de vida del contenedor

```
docker compose stop      # pausa (conserva el contenedor)
docker compose start     # reanuda
docker compose down      # elimina el contenedor (./VaN3Twin y ./results NO se tocan)
docker system df         # espacio usado por imágenes/volúmenes
```

8 Visualización de mensajes y de la simulación

Cuatro herramientas complementarias, de más a menos recomendada para *ver el intercambio de mensajes*: Wireshark (contenido real de cada CAM/DENM), NetAnim (animación de paquetes entre nodos), PyViz (vista interactiva en vivo) y el `vehicle-visualizer` de VaN3Twin (mapa web de vehículos; movilidad, no mensajes).

8.1 Wireshark sobre los .pcap: el contenido de cada mensaje

VaN3Twin transmite mensajes ETSI *reales* (ASN.1 UPER sobre BTP/GeoNetworking), y varios ejemplos ya generan capturas automáticamente mediante `EnablePcap()`: `v2v-simple-cam-exchange-80211p` escribe `v2v-80211p-student-application-*.pcap` y `v2v-emergencyVehicleAlert-80211p` escribe `v2v-EVA-*.pcap` (un fichero por vehículo, en `ns-3-dev/`). Wireshark disecciona la pila completa: GeoNetworking (ethertype 0x8947) → BTP → CAM/DENM con todos sus campos (posición, velocidad, heading, contenedores...).

Ejemplo completo:

Listing 5: Dentro del contenedor

```
# ojo: este ejemplo no admite --sumo-gui (la GUI va fija); visor VNC abierto
./ns3 run "v2v-simple-cam-exchange-80211p --sim-time=30"
cp v2v-80211p-student-application-*.pcap ~/results/
```

Listing 6: En el Mac

```
brew install --cask wireshark
open ~/van3twin-docker/results/v2v-80211p-student-application-0-0.pcap
```

En Wireshark: filtros de visualización útiles: `gnw` (GeoNetworking), `btpb` (BTP-B) e `its` (mensajes CAM/DENM decodificados). El menú *Statistics* → *Flow Graph* genera un diagrama de secuencia del intercambio — lo más parecido a un “visor de mensajes”.

8.2 NetAnim: animación de paquetes entre nodos

NetAnim reproduce (offline) un XML generado por la simulación: nodos moviéndose y cada transmisión como una flecha. El árbol `ns-3` de VaN3Twin ya incluye el módulo `netanim`. **En la imagen actual, el ejemplo `v2v-simple-cam-exchange-80211p` viene parcheado de fábrica** (capa 6b del Dockerfile): genera `v2v-cam-exchange-anim.xml` en cada ejecución sin tocar nada. Para añadir la animación a *otro* ejemplo, estos son los tres cambios (mostrados sobre `src/automotive/examples/v2v-simple-`

Listing 7: Cambio 1: junto al resto de `#include`

```
#include "ns3/netanim-module.h"
```

Listing 8: Cambio 2: justo ANTES de `Simulator::Run()` (línea ~362)

```
AnimationInterface anim ("v2v-cam-exchange-anim.xml");
anim.SetMaxPktsPerTraceFile (500000);
```

Listing 9: Cambio 3: enlazar el módulo `netanim` en `src/automotive/examples/CMakeLists.txt` (bloque del ejemplo, línea ~125). Sin esto el enlace falla con `collect2: error: ld.`

```
build_lib_example(
    NAME v2v-simple-cam-exchange-80211p
    SOURCE_FILES v2v-simple-cam-exchange-80211p.cc
    LIBRARIES_TO_LINK
    ${libcarla}
    ${libautomotive}
    ${libwave}
    ${libtraci}
    ${libnetanim}
)
```

Listing 10: Compilar el ejemplo y ejecutar

```
# dentro del contenedor
./ns3 build                # recompila el ejemplo con la animacin
./ns3 run "v2v-simple-cam-exchange-80211p --sim-time=30"
# en el Mac (visor VNC abierto): abrir el visor, ya instalado en la imagen
docker compose exec van3twin NetAnim
# File > Open -> v2v-cam-exchange-anim.xml (generado en ns-3-dev/)
```

Implicación

El visor NetAnim es una aplicación Qt5 que no existe en los repos de Ubuntu: la imagen la compila durante el build y la deja instalada en `/usr/local/bin/NetAnim`, lista para usar. Muestra “paquetes” genéricos (quién transmite a quién y cuándo), no el contenido ETSI: para el contenido, Wireshark (§8.1). Con muchos vehículos el XML crece rápido; úsalo con simulaciones cortas.

Cómo interpretar la ventana

El lienzo de la pestaña *Animator* es el plano cartesiano del mapa SUMO, en **metros**. Cada punto rojo es un nodo ns-3 (un vehículo), numerado por orden de creación (0..N-1; el *veh1* de SUMO es el nodo 0). Claves de lectura:

- **Vehículos “aparcados” en grupo** (típicamente en una esquina o sobre el origen): son nodos cuyo vehículo aún no ha entrado en la simulación SUMO (o ya salió); TraCI los mantiene fuera del mapa hasta su instante de inserción. Es normal verlos incorporarse al recorrido a medida que avanza el reloj.
- **Flechas entre nodos** durante la reproducción: cada transmisión registrada. En 802.11p los CAM son *broadcast*: un único envío genera una flecha hacia cada receptor al alcance, por lo que la densidad de flechas es un proxy visual de la carga de canal.
- **Cadencia**: los CAM ETSI se emiten entre 10 Hz y 1 Hz según la dinámica del vehículo (reglas de generación ETSI); si un nodo “parpadea” más, es que se mueve/gira más.

Controles útiles

- **Barra superior**: play/pausa; *Pause At* (detiene la reproducción en un instante concreto); deslizador *fast-slow* (intervalo de refresco de la animación: hacia *slow* se ve cada transmisión con calma); barra *Sim time* para saltar a cualquier instante; rejilla (**#** y *Lines*); *Node Size* (súbelo a 2–4 para ver mejor los vehículos); botones *IP/MAC* (etiquetas de direcciones sobre cada nodo) y *T* (trayectorias).
- **Barra izquierda**: lupas de zoom; botón *ID* (mostrar/ocultar números de nodo); icono espiral (círculos de transmisión inalámbrica: dibuja el alcance de cada emisión como onda, muy ilustrativo en V2X); icono de prohibido (oculta las flechas de paquetes si saturan la vista); *(X,Y)* muestra las coordenadas del cursor para medir distancias sobre el mapa.
- **Pestaña Packets**: tabla de todas las tramas (instante, nodo origen → destino y metadatos 802.11/LLC); filtrable por nodo. Es el puente natural hacia Wireshark: aquí ves *cuándo y entre quiénes*, allí el *contenido* ETSI.
- **Pestaña Stats**: contadores y propiedades por nodo.

Consejo operativo: mantén las simulaciones cortas (`-sim-time=30`) — el XML crece rápido con muchos CAMs — y usa *Pause At* + zoom para analizar un evento concreto (p. ej. el instante en que dos vehículos entran en alcance mutuo).

8.3 PyViz: vista interactiva en vivo

PyViz (`-vis`) abre durante la simulación una ventana GTK con la topología en vivo: nodos, movimiento y últimas transmisiones, con pausa e inspección de nodos. Requiere los bindings Python de ns-3.

Advertencia

Estado verificado en esta instalación (arm64, VaN3Twin master, ago-2026): PyViz NO está disponible. Aunque el configure con `-enable-python-bindings` pasa (vía `pybindgen`), los bindings *pregenerados* del ns-3 original no compilan contra las cabeceras que VaN3Twin parchea: fallan `propagation` y `tap-bridge` (excluíbles) y después módulos esenciales del núcleo, que PyViz sí necesita. Es un problema de *upstream* (regenerar los bindings para el árbol parcheado), no de Docker ni de Apple Silicon. Por ello la imagen compila por defecto con `-disable-python`; si DriveX lo arreglara en el futuro, se puede reintentar sin editar nada (hay fallback automático a sin-Python):

```
docker compose build --build-arg NS3_PYTHON_FLAG=--enable-python-bindings
```

Las alternativas equivalentes, ya incluidas y funcionales, son **NetAnim** (§8.2) para la animación y **Wireshark** (§8.1) para el contenido de los mensajes.

8.4 PyViz vs. NetAnim: ¿cuál usar?

Criterio	PyViz	NetAnim
Momento	En vivo , durante la simulación (puede pausarla e inspeccionar nodos).	Offline : reproduce después un XML grabado por la simulación.
Puesta en marcha	No disponible en este fork : los bindings pregenerados no compilan con los parches de VaN3Twin (§8.3).	Visor ya instalado en la imagen; solo 2 líneas de código en el ejemplo.
Interacción	Alta: zoom, pausa, seguir un nodo, ver últimas transmisiones al vuelo.	Media: reproducir, rebobinar, cambiar velocidad, estadísticas y tablas de paquetes.
Reproducibilidad	No deja traza: lo que ves se pierde al cerrar.	El XML es un artefacto: se archiva, se comparte y se reproduce idéntico (útil para papers y docencia).
Escalabilidad	Buena en vivo (no graba nada).	El XML crece rápido con muchos vehículos/CAMs: usarlo en simulaciones cortas.
Contenido mostrado	Topología y transmisiones genéricas (no decodifica ETSI).	Ídem: flechas de paquetes con metadatos básicos.

Regla práctica: **NetAnim** si quieres una animación repetible y barata de conseguir — es la mejor relación esfuerzo/resultado en esta instalación; **PyViz** queda descartado en esta instalación mientras *upstream* no regenere los bindings (§8.3). Y en ambos casos, para el *contenido* de los mensajes (campos del CAM/DENM), la herramienta es Wireshark (§8.1): los animadores muestran *que* se transmite, no *qué* se transmite.

8.5 vehicle-visualizer: mapa web de vehículos

El módulo `src/vehicle-visualizer` muestra los vehículos en vivo sobre un mapa web (Leaflet). Requisitos, verificados en el código del módulo:

- **Node.js en el contenedor**: el modelo C++ comprueba `command -v node` y lanza `node server.js 8080 &`. Ya viene en el Dockerfile actualizado; en contenedores previos: `sudo apt-get install -y nodejs`. (Las dependencias JS, `node_modules`, vienen incluidas en el repo: no hace falta `npm install`.)

- **Puerto 8080 publicado:** ya está en el compose (8080:8080); el socket UDP interno 48110 no necesita publicarse.
- **Activarlo por ejemplo:** los ejemplos que lo soportan exponen el flag `-vehicle-visualizer` (p. ej. `v2v-emergencyVehicleAlert-80211p`, `v2i-*`).
- **Token de Mapbox: opcional.** Sin token, el mapa usa teselas estándar de OpenStreetMap (suficiente). Un token gratuito escrito en `src/vehicle-visualizer/js/mapbox_token` habilita además las capas satélite/híbrida de Mapbox.

```
# dentro del contenedor
./ns3 run "v2v-emergencyVehicleAlert-80211p --sumo-gui=false --vehicle-visualizer=true"
# en el Mac, con la simulación corriendo:
open http://localhost:8080
```

9 Sionna sin GPU: servidor nativo en el Mac

9.1 Análisis: ¿se puede usar Sionna sin GPU NVIDIA?

Sí. El trazado de rayos de Sionna RT no lo hace TensorFlow sino **Mitsuba 3 + Dr.Jit**, que tienen dos backends: CUDA (solo GPUs NVIDIA) y **LLVM (CPU puro)**. Sin CUDA, Sionna cae automáticamente al backend LLVM. El propio script servidor del repo (`src/sionna/sionna_v1_server_script.py`) lo contempla con la opción `-gpu 0` (“*set 0 to use CPU only*”).

La pregunta real es *dónde* ejecutar el servidor. La integración es desacoplada: ns-3 habla con el servidor Sionna por **UDP (puerto 8103)** y la IP es configurable, así que el servidor puede vivir fuera del contenedor. Verificando los wheels publicados en PyPI para las versiones que VaN3Twin soporta (v0.19 y v1.0):

Paquete	Versión (fijada)	macOS arm64	Linux aarch64	Linux x86_64
sionna	1.0.2 (probada por DriveX)	puro Python (depende de los de abajo)		
sionna-rt	1.0.2	puro Python		
mitsuba	3.6.2 (fijada por sionna-rt)	✓	×	✓
drjit	1.0.3 (fijada por sionna-rt)	✓	×	✓
tensorflow	~2.19	✓	✓	✓

Implicación

Consecuencias directas de la tabla:

- **Dentro del contenedor arm64: no** (con las versiones soportadas): mitsuba 3.6.2 no publica wheels Linux aarch64; habría que compilar Mitsuba desde fuente. (Las versiones nuevas — mitsuba 3.9 / sionna 2.x — sí traen aarch64, pero DriveX no las ha probado.)
- **Contenedor amd64 + Rosetta: no:** TensorFlow x86_64 requiere instrucciones AVX, que Rosetta 2 no emula (el proceso muere con *illegal instruction*).
- **Nativo en macOS arm64: sí** — todos los paquetes tienen wheels oficiales. Es la opción recomendada sin GPU, y además usa los núcleos del chip M directamente, sin el peaje de la VM.

9.2 Arquitectura de la solución nativa

La figura 8 muestra las piezas y el flujo de datos: ns-3 publica las posiciones que recibe de SUMO y consulta al servidor los valores de canal calculados por ray tracing.

9.3 Instalación (una sola vez)

Con el contenedor ya construido (§6), ejecutar desde la carpeta de trabajo:

```
./setup-sionna-macos.sh
```

El script (Anexo A) hace, en orden: instala **python@3.12** y **llvm** con Homebrew si faltan; crea el `venv ~/van3twin-sionna` e instala **sionna==1.0.2** con **tensorflow~2.19**; copia `sionna_v1_server_script.py` y la carpeta `scenarios/` desde el contenedor (`docker compose cp`), garantizando que servidor y ns-3 usan exactamente la misma versión del protocolo; y verifica la importación de `drjit/mitsuba/sionna.rt`.

Implicación

Elecciones fijadas por compatibilidad de wheels: **Python 3.12** (techo común entre mitsuba 3.6.2 y TensorFlow 2.19 en arm64), **TF ~2.19** (sionna 1.0.2 exige `numpy<2.0` y excluye TF 2.16/2.17), y **LLVM de Homebrew** exportado vía `DRJIT_LIBLLVM_PATH` (Dr.Jit lo carga en tiempo de ejecución para el backend CPU).

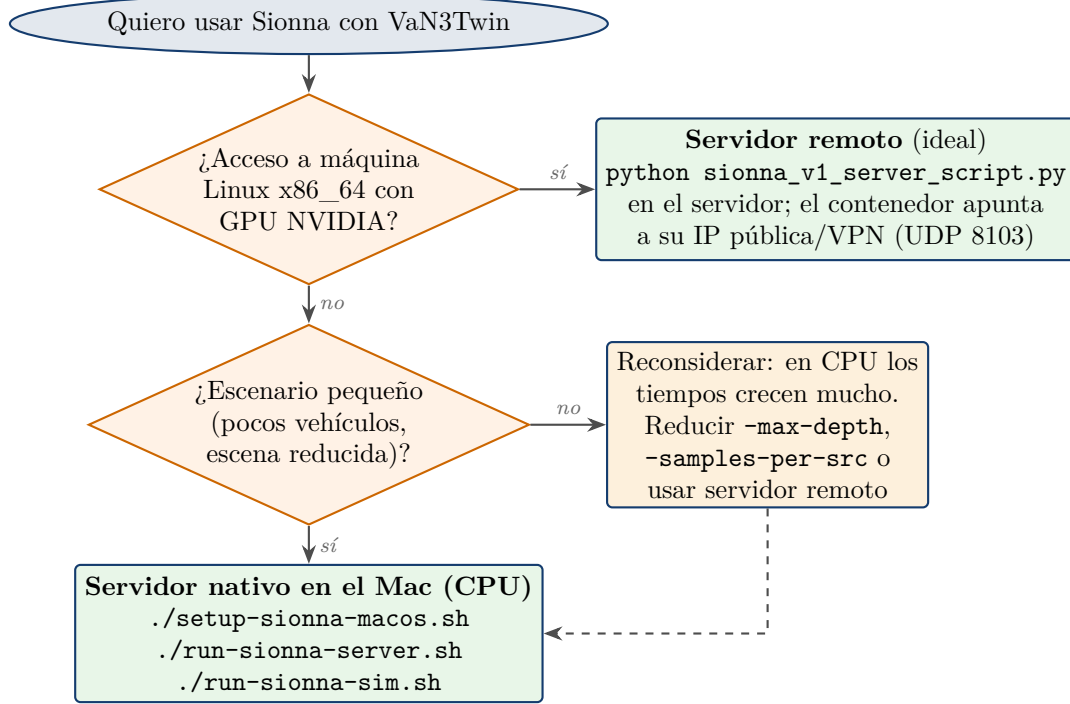


Figura 7: Decisión de despliegue del servidor Sionna cuando el Mac no tiene GPU NVIDIA.

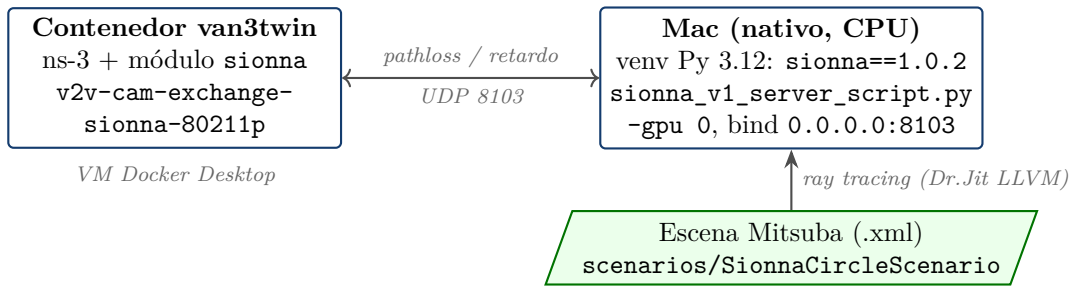


Figura 8: Sionna nativo: ns-3 envía posiciones de vehículos (TraCI/SUMO) y recibe pérdidas de propagación y retardos calculados por ray tracing en la CPU del Mac. No se publica ningún puerto: el contenedor inicia la conexión saliente.

9.4 Ejecución

Listing 11: Terminal 1 (Mac): servidor Sionna en CPU

```
./run-sionna-server.sh          # bind 0.0.0.0:8103, --gpu 0
# opciones tiles en CPU: --verbose --max-depth 3 --samples-per-src 1000
```

Listing 12: Terminal 2 (Mac): simulación en el contenedor

```
open vnc://127.0.0.1:5901      # este ejemplo abre sumo-gui SIEMPRE
./run-sionna-sim.sh
```

El script `run-sionna-sim.sh` (Anexo B) resuelve la IP del host *dentro* del contenedor y lanza:

```
./ns3 run "v2v-cam-exchange-sionna-80211p --sionna=true --sionna-server-ip=$SIONNA_IP"
```

Advertencia

Tres detalles que ahorran horas de depuración:

- El código C++ de VaN3Twin usa `inet_aton()`, que **no resuelve nombres DNS**: pasar `host.docker.internal` como `-sionna-server-ip` falla en silencio. Hay que pasar la IP numérica (el script la resuelve con `getent hosts`).
- El servidor debe arrancarse **sin** `-local-machine`: con ese flag haría bind solo a `127.0.0.1` y el tráfico del contenedor (que llega por la VM) no lo alcanzaría.
- El ejemplo `v2v-cam-exchange-sionna-80211p` tiene `SumoGUI` fijado a `true` en el código (sin flag CLI): requiere la GUI activa — abrir el visor VNC (`vnc://127.0.0.1:5901`, §7.2) — o editar esa línea y recompilar.

La primera vez, el cortafuegos de macOS preguntará si Python puede aceptar conexiones entrantes: aceptar.

9.5 Expectativas de rendimiento en CPU

El servidor mitiga el costo del ray tracing re-trazando solo cuando un vehículo se desplaza más de 3 m o gira más de 90° (`-position-threshold`, `-angle-threshold`) y cacheando pérdidas y retardos. Aun así, en CPU cada re-trazado toma segundos (frente a milisegundos en GPU): es viable para escenas pequeñas como `SionnaCircleScenario` con pocos vehículos, y se degrada rápido al crecer la escena, `-max-depth` o el número de vehículos. Para campañas de simulación grandes, usar la opción de servidor remoto con GPU (figura 7).

10 Solución de problemas

Síntoma	Causa y solución
<code>g++: internal compiler error: Killed (program cc1plus)</code> durante <code>docker compose build</code>	OOM: la VM se quedó sin RAM. Subir memoria en Docker Desktop (§3.2) o bajar <code>cpus</code> y relanzar la build (continúa desde la capa cacheada).
<code>sumo-gui: cannot connect to X server host.docker.internal:0</code> (<i>solo alternativa XQuartz</i>)	XQuartz sin “Allow connections from network clients”, o falta <code>xhost + 127.0.0.1</code> , o XQuartz cerrado. Repetir §3.3 y reintentar — o usar el método VNC por defecto (§7.2).
En el visor VNC las ventanas de SUMO salen sin bordes, inmóviles, sobre fondo negro; o <code>sumo-gui</code> da <code>X Error: BadValue (major 150)</code>	Xvfb arrancado sin gestor de ventanas y/o sin <code>+extension GLX</code> (compose antiguo). Actualizar al compose actual (arranca <code>fluxbox</code> y Xvfb con GLX) o dentro del contenedor: <code>sudo apt-get install -y fluxbox; pkill x11vnc; pkill Xvfb</code> y recrear el contenedor (<code>docker compose up -d</code>).
<code>sumo-gui: cannot connect to X server :99</code> (método VNC)	El Xvfb interno no está corriendo: contenedor creado con un compose antiguo (sin el <code>command</code> que lo arranca) o imagen sin <code>xvfb</code> . Reconstruir la imagen (<code>docker compose build</code>) y recrear el contenedor (<code>docker compose up -d</code>).
<code>X Error: code 161 ... GLXBadContext</code> repetido; la ventana de SUMO abre pero el mapa no se dibuja (<i>solo alternativa XQuartz</i>)	GLX indirecto desactivado en XQuartz. En el Mac: <code>defaults write org.xquartz.X11 enable_iglx -bool true</code> y reiniciar XQuartz. Alternativa infalible (más lenta), en el contenedor: <code>LIBGL_ALWAYS_INDIRECT=0 LIBGL_ALWAYS_SOFTWARE=1 ./ns3 run ...</code>
<code>libGL error: failed to load driver: swrast</code> (y sigue el <code>GLXBadContext</code> aun con <code>LIBGL_ALWAYS_SOFTWARE=1</code>)	Dos causas combinadas: (a) falta <code>libgl1-mesa-dri</code> en el contenedor (<code>sudo apt-get install -y libgl1-mesa-dri mesa-utils</code> ; ya en el Dockerfile actualizado), y (b) <code>LIBGL_ALWAYS_INDIRECT=1</code> sigue definido por el compose — ponerlo <code>=0</code> puede no bastar: hay que hacer <code>unset LIBGL_ALWAYS_INDIRECT</code> . Verificar con <code>glxinfo -B</code> (debe reportar <code>llvmpipe</code>). Además, si XQuartz no expone visuales GLX (<code>iglx</code> inactivo), el modo software contra XQuartz también falla: comprobar en el Mac <code>defaults read org.xquartz.X11 enable_iglx → 1</code> y cerrar XQuartz por completo antes de reabrir.
Los errores GLX persisten con XQuartz haga lo que haga	Abandonar XQuartz y usar la ruta VNC por defecto (§7.2). No depende de GLX del servidor de macOS y funciona siempre.
<code>docker compose up</code> falla con <code>ports are not available ... 127.0.0.1:5900 ... address already in use</code>	El 5900 del Mac lo ocupa el servicio <i>Compartir pantalla</i> /Gestión remota de macOS (o otro VNC). El compose actualizado ya publica <code>127.0.0.1:5901:5900</code> ; conectar a <code>vnc://127.0.0.1:5901</code> . Diagnóstico: <code>sudo lsof -nP -iTCP:5900 -sTCP:LISTEN</code> .
<code>FXApp::openDisplay: unable to open display</code> seguido de <code>Can not connect to sumo via traci ... Connection refused</code>	No es un fallo de TraCI: SUMO murió al no poder abrir la GUI (con VNC: Xvfb no corriendo; con XQuartz: cerrado o sin <code>xhost</code>), y ns-3 no encontró el socket. Verificar la GUI (contenedor arriba y visor VNC abierto) y reintentar, o lanzar el ejemplo con <code>-sumo-gui=false</code> .
La GUI abre pero se cierra / errores libGL	<code>shm_size</code> insuficiente (subirlo a <code>2gb</code> en el compose) o, en la alternativa XQuartz, falta <code>LIBGL_ALWAYS_INDIRECT=1</code> .
Please do NOT run this script as root	Se ejecutó <code>sandbox_builder.sh</code> manualmente como root (p.ej. con <code>docker exec -u root</code>). Usar siempre el usuario <code>vanet</code> .
Tras <code>docker compose build</code> , el contenedor	La carpeta <code>./VaN3Twin</code> (bind mount) tapa la imagen

11 Referencias

- Repositorio VaN3Twin: <https://github.com/DriveX-devs/VaN3Twin>
- Documentación oficial (ms-van3t): <https://ms-van3ts-documentation.readthedocs.io/en/master/>
- ns-3 fork 5G-LENA (CTTC): <https://gitlab.com/cttc-lena/ns-3-dev> (etiqueta ns-3-dev-v2x-v0.2) y módulo nr (rama nr-v2x-dev)
- SUMO: <https://sumo.dlr.de>
- Docker Desktop para Mac: <https://docs.docker.com/desktop/setup/install/mac-install/>
- XQuartz: <https://www.xquartz.org>
- Sionna — instalación y requisitos (CPU/LLVM): <https://nvlabs.github.io/sionna/installation.html>
- Wheels de Mitsuba/Dr.Jit (verificación de arquitecturas): <https://pypi.org/project/mitsuba/>, <https://pypi.org/project/drjit/>

A setup-sionna-macos.sh (Sionna nativo, instalación)

```
#!/bin/bash
# =====
# setup-sionna-macos.sh -- Instala el servidor NVIDIA Sionna v1.0 NATIVO en el
#                               Mac (Apple Silicon, CPU sin GPU NVIDIA) para usarlo
#                               con el contenedor VaN3Twin.
#
# Qué hace:
# 1. Instala (si faltan) python@3.12 y llvm vía Homebrew.
# 2. Crea un venv en ~/van3twin-sionna con sionna==1.0.2 (la versión
#    soportada por VaN3Twin) + TensorFlow arm64.
# 3. Copia el script servidor y los escenarios desde el contenedor.
#
# Requisitos: Homebrew, contenedor van3twin construido (docker compose up -d).
# Ejecutar DESDE la carpeta que contiene docker-compose.yml: ./setup-sionna-macos.sh
# =====
set -e

SIONNA_DIR="$HOME/van3twin-sionna"

# --- 0. Comprobaciones previas -----
if [ ! -f "docker-compose.yml" ]; then
    echo "ERROR: ejecuta este script desde la carpeta que contiene docker-compose.yml"
    exit 1
fi
if ! command -v brew >/dev/null 2>&1; then
    echo "ERROR: se requiere Homebrew (https://brew.sh)"
    exit 1
fi

# --- 1. Dependencias del Mac -----
# Python 3.12: mitsuba 3.6.2 (fijado por sionna-rt 1.0.2) publica wheels
# macOS_arm64 hasta cp313, pero TensorFlow 2.19 llega hasta cp312.
if ! command -v python3.12 >/dev/null 2>&1; then
    echo "[setup] Instalando python@3.12..."
    brew install python@3.12
fi
# LLVM: lo necesita Dr.Jit (backend CPU del ray tracer, al no haber CUDA).
if ! brew --prefix llvm >/dev/null 2>&1 || [ ! -e "$(brew --prefix llvm)/lib/libLLVM.dylib" ]; then
    echo "[setup] Instalando llvm..."
    brew install llvm
fi

# --- 2. Entorno virtual con Sionna v1.0 -----
mkdir -p "$SIONNA_DIR"
cd "$SIONNA_DIR"
if [ ! -d venv ]; then
    python3.12 -m venv venv
fi
source venv/bin/activate
pip install --upgrade pip
# sionna 1.0.2 fija: sionna-rt==1.0.2 -> mitsuba==3.6.2 + drjit==1.0.3
# (todos con wheels macOS_arm64) y numpy<2.0.
# TF ~2.19: compatible con numpy 1.26.x y con Python 3.12 en arm64.
pip install "sionna==1.0.2" "tensorflow==2.19.0"

# --- 3. Servidor y escenarios desde el contenedor -----
cd - >/dev/null
docker compose up -d
docker compose cp \
    van3twin:/home/vanet/VaN3Twin/ns-3-dev/src/sionna/sionna_v1_server_script.py \
    "$SIONNA_DIR/"
docker compose cp \
```

```
van3twin:/home/vanet/VaN3Twin/ns-3-dev/src/sionna/scenarios \  
"$SIONNA_DIR/scenarios"  
  
# --- 4. Verificación -----  
export DRJIT_LIBLLVM_PATH="$(brew --prefix llvm)/lib/libLLVM.dylib"  
"$SIONNA_DIR/venv/bin/python" - <<'EOF'  
import drjit, mitsuba, sionna.rt  
print(f"[setup] OK -> drjit {drjit.__version__} | mitsuba {mitsuba.__version__} | sionna-rt  
↳ importado")  
print("[setup] Backend CPU (LLVM) listo; no se requiere GPU NVIDIA.")  
EOF  
  
echo ""  
echo "[setup] Instalación completa en $SIONNA_DIR"  
echo "[setup] Arranca el servidor con: ./run-sionna-server.sh"
```

B run-sionna-server.sh y run-sionna-sim.sh (Sionna nativo, ejecución)

```
#!/bin/bash
# =====
# run-sionna-server.sh -- Arranca el servidor Sionna v1.0 NATIVO en el Mac,
#                          en modo CPU (sin GPU NVIDIA), escuchando en UDP/8103.
#
# El contenedor VaN3Twin le enviará las posiciones de los vehículos y le
# pedirá pathloss/retardo por ray tracing (ver run-sionna-sim.sh).
#
# Notas:
# * SIN --local-machine el script hace bind a 0.0.0.0 ("external server"),
#   necesario porque el tráfico del contenedor llega vía la VM de Docker.
# * --gpu 0 = solo CPU (backend LLVM de Dr.Jit). Documentado por los
#   propios autores del script.
# * macOS puede preguntar si permites que Python acepte conexiones
#   entrantes: acepta.
#
# Uso: ./run-sionna-server.sh [args extra del servidor]
#   p. ej.: ./run-sionna-server.sh --verbose --max-depth 3
# =====
set -e

SIONNA_DIR="$HOME/van3twin-sionna"

if [ ! -f "$SIONNA_DIR/sionna_v1_server_script.py" ]; then
    echo "ERROR: no existe $SIONNA_DIR. Ejecuta primero ./setup-sionna-macos.sh"
    exit 1
fi

# Dr.Jit necesita libLLVM para el backend CPU
export DRJIT_LIBLLVM_PATH="$(brew --prefix llvm)/lib/libLLVM.dylib"

cd "$SIONNA_DIR"
source venv/bin/activate

echo "[sionna] Servidor CPU-only en UDP/8103 (Ctrl+C para detener)"
echo "[sionna] Escenario por defecto: escenarios/SionnaCircleScenario/scene.xml"
exec python sionna_v1_server_script.py \
    --gpu 0 \
    --port 8103 \
    "$@"
```

```
#!/bin/bash
# =====
# run-sionna-sim.sh -- Lanza dentro del contenedor la simulación de ejemplo
#                      v2v-cam-exchange-sionna-80211p apuntando al servidor
#                      Sionna que corre NATIVO en el Mac (UDP/8103).
#
# Requisitos:
# 1. Servidor Sionna corriendo en el Mac: ./run-sionna-server.sh
# 2. GUI disponible: visor VNC abierto (open vnc://127.0.0.1:5901).
#   Este ejemplo abre sumo-gui SIEMPRE: SumoGUI está fijado a true en su
#   código; para correr sin GUI, edita la línea
#   'SetAttribute ("SumoGUI", ...)' del ejemplo y recompila con ./ns3 build.
#
# Detalle técnico: el código C++ de VaN3Twin usa inet_aton(), que NO resuelve
# nombres DNS, por lo que hay que pasar la IP numérica del host. La resolvemos
# dentro del contenedor a partir de host.docker.internal.
# =====
set -e
```

```
docker compose up -d

docker compose exec van3twin bash -c '
    SIONNA_IP=$(getent hosts host.docker.internal | cut -d" " -f1)
    if [ -z "$SIONNA_IP" ]; then
        echo "ERROR: no se pudo resolver host.docker.internal"
        exit 1
    fi
    echo "[sim] Servidor Sionna (Mac): $SIONNA_IP:8103"
    ./ns3 run "v2v-cam-exchange-sionna-80211p --sionna=true --sionna-server-ip=$SIONNA_IP"
,
```